Proposal: Minimizing Network Bandwidth for Local Testing

Version: 0.1.0
Status: Draft
Style Guide: style-guide--web-ready-unrendered-markdown-using-apa-7

Abstract

The local security scan that replaced CodeQL in uc-cboard downloads roughly 165 MB every time it runs, almost all of it identical to the previous run. This proposal measures where that bandwidth goes, examines nine ways to reduce it, and compares them on first-run cost, steady-state cost per run, update cost, complexity, and reuse across projects. It recommends a combination in which each tool version crosses the network once, into a verified local archive kept by an OSAT manager, and every scan starts from a versioned, prepared Incus image built from that archive with no network access. In steady state a scan then uses well under 1 MB, the dependency vulnerability lookup, and updates become deliberate operator actions rather than a side effect of running tests.

Sources and acknowledgements

This proposal applies the standing directive *Collaboration: Energy Conservation in AI Collaborations* (Steel, 2026a), which treats compute and bandwidth as finite resources and asks that avoidable intensive operations not be repeated. The manager design it recommends follows the lifecycle, archive and `PROVENANCE` contract defined in *Proposal: osat-manager-python Layout, Lifecycle and Aliases* (Steel, 2026b). Incus behaviour is taken from the Incus documentation (Linux Containers, n.d.-a). Measurements were taken on 2026-10-04 from the first run of `make sast`, documented in *Local Security Scan With Incus* (Steel, 2026c), and from HTTP `Content-Length` headers of the pinned release files. The analysis was prepared with AI assistance from Claude Opus 5.5 (Anthropic).

Design principle

Bandwidth is spent once per version, never once per run. A file that has been downloaded and verified is kept, and every later use reads the local copy. Network access happens only when an operator decides to update, and an update fetches only what changed.

Context

`make sast` starts a throwaway Incus container from `images:debian/12`, copies the working tree in, and runs `scripts/sast/inside.sh`. That script installs base packages with `apt`, then downloads and verifies Node.js, OSV-Scanner, Gitleaks and Opengrep, installs ESLint and its plugins from the npm registry, and runs the scanners. When the scan ends, the container is deleted, and everything it downloaded goes with it. The next run downloads the same files again.

The same pattern will apply to any further tests run in Incus, for example the Jest unit tests and Playwright end-to-end tests, which would add a full `yarn install` of 2,084 packages and browser binaries to every run if built the same way.

Where the bandwidth goes

The table below lists what one scan run downloads. Sizes marked measured come from the release files' `Content-Length` headers or from a direct download of the file; sizes marked estimated are not yet measured end to end.

Item	Source	Size	Basis
OSV-Scanner 2.6.0 binary	GitHub releases	54.9 MB	Measured
Opengrep 1.30.0 binary	GitHub releases	44.4 MB	Measured
Node.js 22.14.0 tarball	nodejs.org	28.5 MB	Measured
Gitleaks 8.30.1 tarball	GitHub releases	7.8 MB	Measured
ESLint, plugins and dependencies	npm registry	about 10 MB	Estimated; `typescript` alone is 4.4 MB and `eslint` 0.6 MB, measured
Debian package indexes	deb.debian.org	about 10 MB	Estimated; the main `Packages.xz` alone is 8.8 MB, measured
`curl`, `git`, `xz-utils` and dependencies	deb.debian.org	about 10 MB	Estimated
Checksum files	nodejs.org, GitHub	under 0.1 MB	Measured
Vulnerability lookup for `yarn.lock`	api.osv.dev	under 1 MB	Estimated
Total per run		**about 165 MB**	

At ten scans a working day this is about 1.65 GB a day for one developer, all of it repeated.

A hidden cost: cached image refresh

The base image itself, `images:debian/12`, is 106 MiB in the local image store. It is not downloaded on each run, because Incus caches it. However, an image cached while creating an instance is marked for automatic update when `images.auto_update_cached` is true, which is the default, and the daemon checks for newer versions of every such image on startup and every `images.auto_update_interval`, six hours by default (Linux Containers, n.d.-a; Linux Containers, n.d.-b). The `images:` server publishes new builds daily; the cached Debian image on this machine carries a build date of 2026-10-04. Each cached image can therefore cost up to one full image download a day whether or not any test runs. This machine currently holds four cached images totalling about 480 MiB.

Options examined

Each option is described with what it saves, what it costs, and its risks. They are not mutually exclusive; the recommendation combines several.

Host-side versioned tool cache

Each verified download is stored on the host under a versioned path, for example `~/.cache/uc-cboard-sast/opengrep/1.30.0/opengrep_manylinux_x86`, beside its checksum. Before downloading, the runner looks in the cache; on a hit it pushes the cached file into the container with `incus file push`, or attaches the cache read-only as a disk device (Linux Containers, n.d.-c). npm packages are handled the same way by archiving a prepared `node_modules` tree as one tarball, built once from a lock file.

Saves: the 135 MB of release binaries and the npm packages on every run after the first.

Costs: about 150 MB of host disk per tool set version. The runner gains cache logic. `apt` traffic is not removed, because the container still installs packages.

Risks: a cache without verification on read can serve a corrupted file. Every read must re-check the stored checksum, as manage-python does when restoring from its archive (Steel, 2026b).

Checking for the latest version cheaply

Knowing whether a newer version exists does not require downloading it. GitHub's REST API returns a release's metadata in a few kilobytes, and a conditional request that sends the previous response's `ETag` in `If-None-Match` returns `304 Not Modified` with no body when nothing has changed (GitHub, n.d.). Node.js and npm offer equivalent metadata endpoints.

Saves: turns "download to find out" into a request of a few kilobytes, or a few hundred bytes when unchanged.

Costs: one request per tool per check. If checks run on every scan, they add up to little but are still wasted when nothing changes.

Risks: the check must stay a check. Automatically installing whatever is newest makes runs non-reproducible and puts unreviewed tool versions into the security gate. The operator decides when to install.

An OSAT manager for the test tool set

An OSAT manager, provisionally `osat-manager-sast` run as `manage-sast`, installs the scan tools into user space with the collection's lifecycle: `--install`, `--switch`, `--status`, `--remove`, and an archive of verified release files with `PROVENANCE` records (Steel, 2026b). The tool set is described by a lock file in each project, for example `scripts/sast/toolset.lock`, naming each tool's version, release asset and SHA-256. `manage-sast --check` performs the conditional version checks described above and reports what is newer; `manage-sast --install` fetches only the tools whose versions are not yet in the archive. Restoring from the archive needs no network, as with manage-python's offline restore.

Saves: the same per-run bandwidth as a host-side tool cache, but shared across every project and every Incus instance on the machine, and across machines when the archive is copied.

Costs: a new manager to write and maintain. Most of the code exists in manage-python and can be shared: download, verify, archive, `PROVENANCE`, owner-only files, sandbox mode, and the refusal to run as root.

Risks: scope creep into a general package manager. The manager should manage a declared tool set of release binaries and prepared tarballs, not resolve dependencies.

Versioned prepared images

Incus can turn an instance into a reusable local image with `incus publish`, give it an alias, and attach properties to it (Linux Containers, n.d.-d). Two images are built and versioned:

- **Base image**, for example alias `uc-sast/base/2026-10-04`: Debian with all updates applied and the base packages (`ca-certificates`, `curl`, `git`, `xz-utils`) installed, and no test tools.
- **Tools image**, for example alias `uc-sast/tools/1.0.0`: the base image plus the tool set installed under `/opt/sast`, with the tool set's lock file hash recorded as an image property.

A scan launches from the tools image. Launching from a local image downloads nothing, and on this machine's btrfs storage pool the new instance is a copy-on-write clone, so it is also fast and uses little extra disk. The tools image is rebuilt only when the tool set changes, starting from the current base image, and the base image is rebuilt only when the operator refreshes the operating system.

Incus snapshots were considered as the versioning mechanism: a stopped "golden" instance with snapshots named `base-2026-10-04` and `tools-1.0.0`, copied with `incus copy golden/tools-1.0.0 scan-run`. Snapshots work, but they are tied to one instance and cannot be listed, exported or shared as easily as images. Published images can be exported to a file with `incus image export` and imported on another machine, which carries the whole prepared environment across without any downloads.

Saves: all per-run downloads, including `apt`, Node.js and npm packages. Only the OSV lookup remains.

Costs: host disk for each retained image; Incus images are complete, not layered, so each tools image includes its base. Building an image takes a few minutes and is done only on change. A retention policy, for example the current and previous version of each, keeps disk use bounded.

Risks: a stale base image misses security updates. The base image's date is visible in its alias, the runner warns when it is older than the agreed refresh interval, and refreshing is one command.

Images built from the archive with no network

Prepared images alone move downloads from every run to every image build. Combined with an OSAT manager, the tools image is built from the local archive: the build instance uses a profile with no network device, the tool files are pushed in from the archive, and the build therefore cannot download anything even by accident. Only the base image refresh uses the network, for `apt`.

Saves: makes image builds network-free, so a tool set change costs only the download of the changed tool, once.

Costs: the archive must hold everything the tools image needs, including the Node.js tarball and the prepared npm `node_modules` tarball.

Risks: none beyond those of the OSAT manager and the prepared images. A build that tries to reach the network fails loudly, which is the intended behaviour.

Local caching proxies

A caching proxy on the host serves repeated downloads locally: apt-cacher-ng for Debian packages (Bloch, n.d.), and Verdaccio as an npm registry cache (Verdaccio, n.d.).

Saves: repeated `apt` and npm downloads for every container on the machine, including ones unrelated to this project.

Costs: two long-running services to install, secure, update and store data for. Containers must be configured to use them. GitHub release binaries are not covered without a general HTTP cache, and HTTPS downloads cannot be cached by a transparent proxy.

Risks: a proxy that is down blocks every build. A proxy keeps serving a cached file without the end-to-end checksum verification the archive gives.

An offline vulnerability database

OSV-Scanner can download its vulnerability database and scan offline (Google, n.d.). The npm database alone is a 207 MiB download, measured on 2026-10-04, against well under 1 MB for an online lookup of `yarn.lock`.

Saves: network access during the scan, which matters on an air-gapped or metered machine.

Costs: each database refresh costs more than two hundred online scans. Refreshing rarely saves bandwidth but reports stale vulnerability data.

Risks: stale data gives a false pass. For a connected developer machine, online lookups are both cheaper and more current.

Controlling Incus image refresh

Each image in the store has an `auto_update` property that can be changed per image (Linux Containers, n.d.-a). Turning it off for the base image this project uses, and for prepared images, which have no source server to update from anyway, removes the background downloads described in the Context. `images.auto_update_cached` can also be set to false server-wide, but that changes the behaviour of every cached image on the machine and is the operator's decision, not this project's.

Saves: up to one image download a day per cached image, independent of how often tests run.

Costs: the base image no longer updates silently, so freshness becomes the operator's responsibility, handled by the base image refresh described under versioned prepared images.

Risks: forgetting to refresh. The age warning described under versioned prepared images addresses this.

Running less

Some runs can be skipped outright. If `yarn.lock` and the OSV-Scanner version are unchanged since the last scan of the same day, the dependency lookup's answer is unlikely to have changed. A Gitleaks history scan only needs commits it has not yet seen. ESLint and Opengrep can be limited to changed files during development, with full scans before a release.

Saves: compute on every run, and the remaining OSV traffic.

Costs: state about previous runs must be kept and trusted.

Risks: skipped checks hide new findings. This option is a refinement once the larger savings are in place, not a substitute for them.

Comparison

Costs are for one developer machine and the current scan tool set. "Per run" is the steady state after the first run.

Option	First run	Per run	Tool update	Complexity	Works offline	Reused across projects
Current scan	165 MB	165 MB	none needed	low	no	no
Host tool cache	165 MB	about 20 MB (`apt`)	changed tool only	low	no	no
Cheap version check	n/a	a few KB if checked each run	n/a	low	n/a	yes
OSAT manager	165 MB	about 20 MB (`apt`)	changed tool only	medium	restore yes	yes
Prepared images	165 MB	under 1 MB	rebuild with download	medium	yes, except OSV	per project
Images from the archive	165 MB	under 1 MB	changed tool only	medium	yes, except OSV	yes
Caching proxies	165 MB	GitHub binaries still fetched	changed tool only	high	partly	yes
Offline OSV database	207 MiB extra	0	207 MiB per refresh	low	yes	yes
Image refresh control	0	0	n/a	low	n/a	yes
Running less	n/a	lower	n/a	medium	n/a	no

Recommendation

A verified archive kept by an OSAT manager

Build `osat-manager-sast` as the single place where scan and test tools cross the network. It downloads each tool version once, verifies it against the published checksum, archives it with a `PROVENANCE` record, and restores from the archive with no network. Updates are operator actions: `manage-sast --check` reports newer versions using conditional requests, and `manage-sast --install` fetches only what is new. Tool versions move out of `inside.sh` into `scripts/sast/toolset.lock`, so a version change is a reviewed commit.

This makes the archive, not the network, the source of every tool, for this project and for any other project on the machine that declares the same tools.

Versioned prepared images built from the archive

Keep two versioned local images per tool set: a base image refreshed on a fixed interval, and a tools image built from the base and the archive in an instance with no network device. `make sast` launches from the tools image whose recorded lock file hash matches `toolset.lock`. If none exists, it builds one from the archive; if the archive lacks a tool, it stops and tells the operator to run `manage-sast --install`, rather than downloading on its own.

Turn off `auto_update` on the base image this project uses, and refresh it on a stated interval instead. Weekly is suggested as a balance between security updates and bandwidth; the runner warns when the base image is older than the interval.

Keep OSV-Scanner's online lookup rather than the offline database: it is the only per-run traffic, under 1 MB, and gives current vulnerability data.

Expected effect

Measure	Today	With both recommendations
Per scan run	about 165 MB	under 1 MB
Ten runs a day	about 1.65 GB	under 10 MB
Tool version change	165 MB on every following run	the changed tool only, once
Base image freshness	up to one 106 MiB download a day per cached image, unplanned	one base refresh a week, planned
A second machine	full downloads	`incus image export` and copy, no downloads

These figures are estimates built on the measurements above. They should be measured again after implementation, for example with the instance network counters that `incus info` reports.

A smaller first step

The manager and the images are a larger piece of work. Until they exist, a host-side tool cache can be added to `run-in-incus.sh` with modest effort, using the same archive layout and `PROVENANCE` format that `osat-manager-sast` will use, so that the cache becomes the manager's archive without moving any files. Together with turning off `auto_update` on the base image, this removes about 135 MB of every run and the background image refreshes immediately.

Alternatives considered

Caching proxies as the primary mechanism. Not recommended. Two services add more to run and secure than they save over a verified archive, and they do not cover GitHub release binaries over HTTPS. They remain useful for an operator who runs many unrelated containers.

Snapshots instead of published images. Not recommended as the versioning mechanism, because snapshots belong to one instance and are harder to list, export and share. They remain useful while building an image.

Mounting the archive into each scan instance instead of using prepared images. Considered as a simpler form of the host-side tool cache. It removes the tool downloads but not `apt`, and shared disk devices behave differently in containers and virtual machines. Prepared images remove all downloads and behave the same in both.

Always installing the latest tool versions. Rejected. It makes runs non-reproducible, puts unreviewed tools into a security gate, and spends bandwidth on every release whether or not anyone needs it.

Open questions

- The manager's name, `osat-manager-sast`, is provisional. A broader name, for example `osat-manager-testtools`, would suit Jest, Playwright browsers and other test tools as well as scanners.
- Whether Playwright browser binaries and the project's own `yarn install` should go through the same archive and images; they are larger than the scan tools and would benefit most.
- The base image refresh interval, suggested here as weekly.
- Whether `images.auto_update_cached` should be turned off server-wide on this machine, which affects every Incus instance, not only this project's.

License

This document, *Proposal: Minimizing Network Bandwidth for Local Testing*, by **Christopher Steel**, with AI assistance from **Claude Opus 5.5 (Anthropic)**, is licensed under the GNU General Public License v3.0 or later.

Resources

Project documents

- Collaboration: Energy Conservation in AI Collaborations
- Proposal: osat-manager-python Layout, Lifecycle and Aliases
- Local Security Scan With Incus

Incus

- About images
- Server configuration
- Disk devices
- Create images

Downloads and caching

- GitHub REST API best practices, conditional requests
- OSV-Scanner offline mode
- apt-cacher-ng

- Verdaccio

References

Bloch, E. (n.d.). *Apt-Cacher NG*. Retrieved October 4, 2026, from https://www.unix-ag.uni-kl.de/~bloch/acng/
Return to citation

GitHub. (n.d.). *Best practices for using the REST API*. GitHub Docs. Retrieved October 4, 2026, from https://docs.github.com/en/rest/using-the-rest-api/best-practices-for-using-the-rest-api
Return to citation

Google. (n.d.). *Offline mode*. OSV-Scanner documentation. Retrieved October 4, 2026, from https://google.github.io/osv-scanner/usage/offline-mode/
Return to citation

Linux Containers. (n.d.-a). *About images*. Incus documentation. Retrieved October 4, 2026, from https://linuxcontainers.org/incus/docs/main/image-handling/
Return to citation

Linux Containers. (n.d.-b). *Server configuration*. Incus documentation. Retrieved October 4, 2026, from https://linuxcontainers.org/incus/docs/main/server_config/
Return to citation

Linux Containers. (n.d.-c). *Type: disk*. Incus documentation. Retrieved October 4, 2026, from https://linuxcontainers.org/incus/docs/main/reference/devices_disk/
Return to citation

Linux Containers. (n.d.-d). *How to create images*. Incus documentation. Retrieved October 4, 2026, from https://linuxcontainers.org/incus/docs/main/howto/images_create/
Return to citation

Steel, C. (2026a). *Collaboration: Energy conservation in AI collaborations* (Version 0.2.0) [Standing directive]. uc-cboard repository, `en/docs/automa/ai-collaboration/defaults/`.
Return to citation

Steel, C. (2026b). *Proposal: osat-manager-python layout, lifecycle and aliases* (Version 0.7.0) [Proposal]. osat-manager-python repository, `en/docs/`.
Return to citation

Steel, C. (2026c). *Local security scan with Incus* (Version 0.1.0) [Guide]. uc-cboard repository, `en/docs/guides/devops/`.
Return to citation

Verdaccio. (n.d.). *What is Verdaccio?* Verdaccio documentation. Retrieved October 4, 2026, from https://verdaccio.org/docs/what-is-verdaccio
Return to citation

Changelog

Version	Status	Notes
0.1.0	Draft	Initial draft: measured per-run bandwidth of the local scan, nine options examined and compared, recommendation of a verified archive kept by an OSAT manager plus versioned prepared Incus images, with a smaller first step